Tradelink Technical Specification

Field	Value
Document	Technical Specification Document (TSD), as built
Product	Tradelink, with Tradelink AI
Version	1.1
Date	4 November 2026
Status	Draft for review
Owner	Janya Mali
Code baseline	`develop` at commit `9250e98` and 17 applied database migrations (latest `20261104040939`)

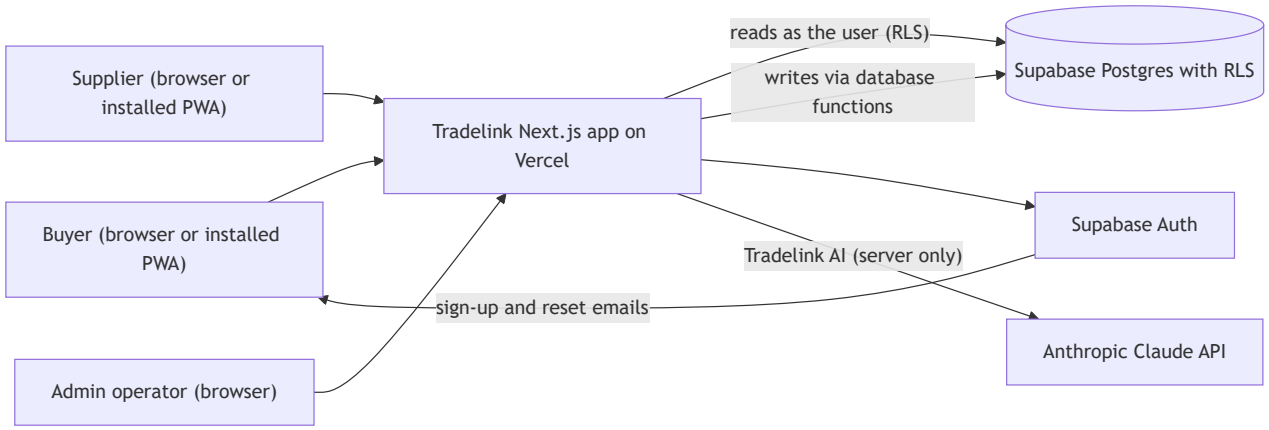
Contents

1. Overview
2. Technology choices
3. Considered but not used
4. Architecture
5. Key flows
6. Data model
7. Security
8. Testing
9. Delivery and operations
10. Running cost
11. Decision log
12. Known technical debt and possible defects

- Appendix A. Differences from older documents
- Document history

Purpose, audience and scope. This document explains how Tradelink is built, what technology it uses and why, and what was left out and why. It is written for engineers who maintain the code. It covers the Next.js app (`app/`, `src/`), the Supabase database (`supabase/migrations/`), the Claude integration, tests, deployment and running cost. Requirements are in the Functional Specification. Every statement comes from the code, migrations, tests or an ADR; where a reason was not recorded it is marked **Inferred rationale**.

1. Overview



- **The browser never talks to Supabase or Claude directly.** Every read and write runs on the server, as the signed-in user.
- **One responsive app** serves phone and laptop (layout switches at 760 px) and can be installed as a PWA.
- **Repository layout:** `app/<role>/` routes, `src/features/` actions and queries, `src/domain/` pure rules, `src/data/` Supabase clients and `callRpc`, `src/integrations/claude/` the AI client, `src/ui/` design-system components.

2. Technology choices

Versions are from `package.json`.

Technology	Version	Why it is used
Next.js (App Router)	16.4.2	Server components and server actions keep keys, data access and AI on the server; one codebase for phone and laptop; Vercel deploys it natively (ADR 0002)
React	19.3.1	Comes with Next.js; `useActionState` and transitions handle form saves
TypeScript (strict)	6.0.3	Catches errors at build time; `any` is banned by lint
Tailwind CSS, with design tokens	4.3.3	Utility classes generated from the design system's `tokens.json`; one token file drives light and dark mode (ADR 0015)
Supabase: Postgres, Auth, RLS	supabase-js 2.118.0, ssr 0.12.7	Real transactions and row locks for stock; Row Level Security for tenancy; managed sign-in and password reset (ADR 0003)
`security definer` database functions	—	Every write checks role, state, stock, audit and notification in one transaction; there are no write policies (ADR 0004)
Claude Sonnet 5.5 via `@anthropic-ai/sdk`	0.135.0, model `claude-sonnet-5-5`	Tradelink AI: tool use for reading and proposing actions, streaming replies, prompt caching (ADR 0008)
Zod	4.6.7	One schema validates action input, route bodies and AI tool input

Technology	Version	Why it is used
Vitest	4.1.12	Fast unit tests for pure domain rules and feature logic
Playwright	1.56.1	End-to-end tests on desktop Chrome and a Pixel 7 profile; also captures documentation screenshots
Vercel	Hobby plan	Git-connected deploys: `develop` previews, `main` production
`web-push`	3.6.7	Installed for push notifications; not switched on (section 3)

Design rules that follow from these choices:

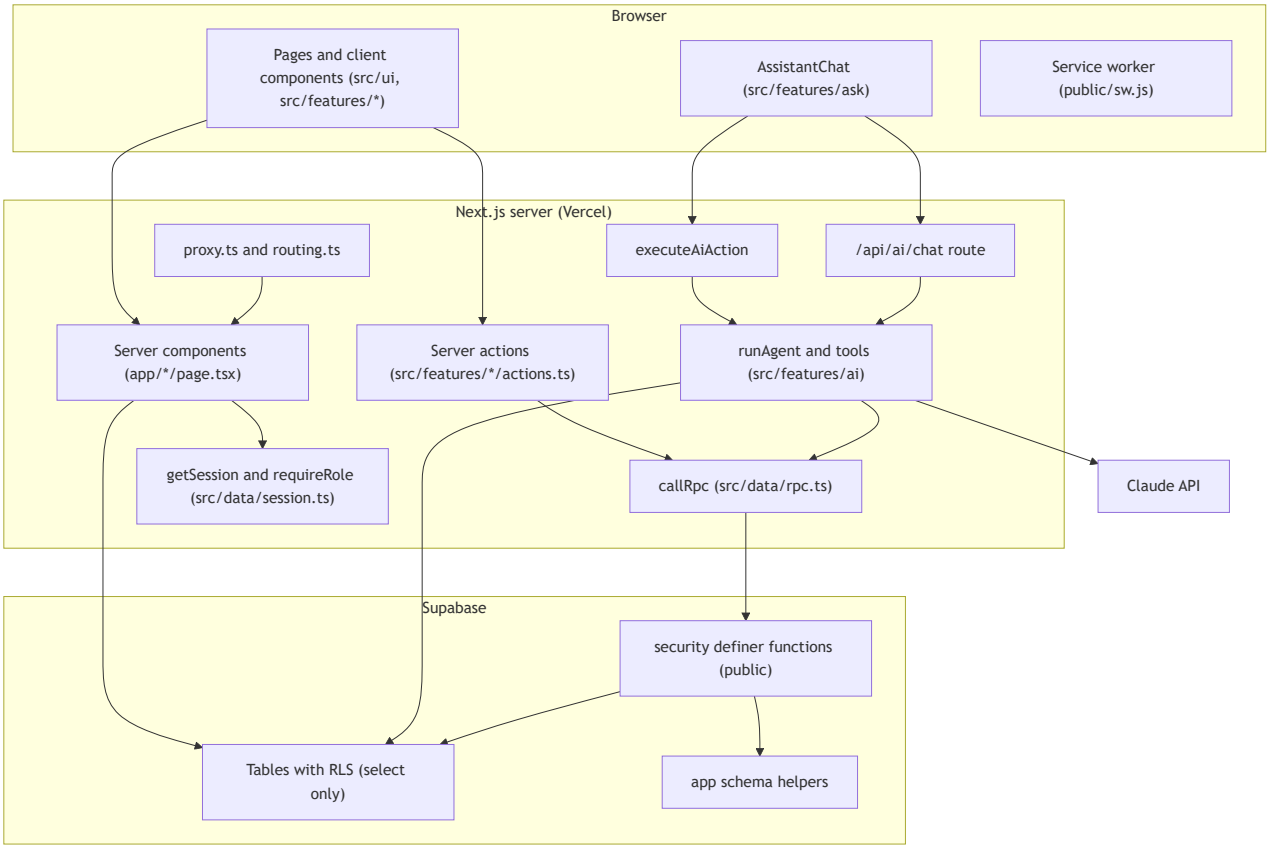
- Reads use the user's Supabase session, so RLS decides what is visible.
- Writes go only through database functions called with `callRpc`; critical writes take an idempotency key (`p_key`).
- Money is integer cents in TypeScript and `numeric(12,2)` in Postgres; times use Asia/Singapore (ADR 0019).
- The AI never writes on its own: every change it proposes needs a person's tap.

3. Considered but not used

Option	Status	Why not (limitation or decision)
Native iOS and Android apps	Not used	One responsive PWA covers phone and laptop with one codebase; users already work in a browser or WhatsApp (ADR 0016)
Stripe or any payments	Not used	Owner decision, 4 November 2026: no payments or subscription billing
Email provider (Resend)	Not configured	No provider account or key; only Supabase Auth's own sign-up and reset emails are sent (ADR 0012 is not built)
Push notifications	Built, not switched on	Plumbing exists (`web-push`, `/api/push/*`, service worker), but no opt-in button and no sender call it; VAPID keys are not set in Vercel
Scheduled jobs (`pg_cron`)	Not applied	The runner and `supabase/migrations/pending_ai_job_cron.sql` exist; the schedule waits for the server secret key in Vercel. Nothing in the live app needs jobs
Supabase branching for a test database	Not available	Needs the Supabase Pro plan; the attempt failed on the Free plan, so tests that write were not run
CI (GitHub Actions)	None	Checks run locally with `npm run check` before pushing. Inferred rationale: a one-developer sprint
Coverage tooling	None	No coverage package is installed; coverage is not measured
Content Security Policy	None	Other headers are set in `next.config.ts`; a CSP was planned in ADR 0010 but not added
Two-factor sign-in	Removed	Owner decision for a working demo with self sign-up (ADR 0024)
Invoice upload and checks (OCR)	Not built	Out of the sprint scope; only an unused `invoice` problem type remains

Option	Status	Why not (limitation or decision)
Rules-based order reader in the live path	Not used	`readOrderWithClaude` with grounding and a rules fallback exists but has no UI caller; the live assistant uses tools instead
AI evaluation set	None	No `evals/` folder; AI behaviour is covered by unit tests with a mocked SDK

4. Architecture



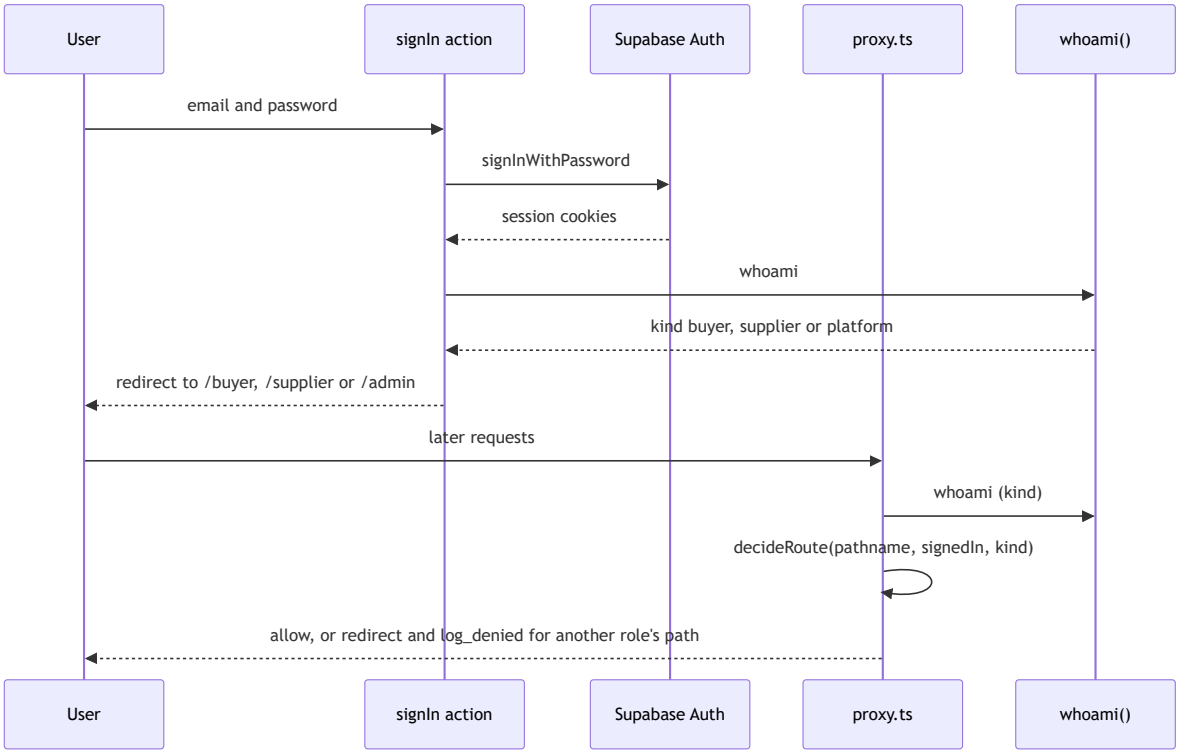
- **Server and client.** Server modules import `server-only`, so a client import fails the build. Client components never hold keys; they call server actions or `/api/ai/chat`.
- **Rendering.** Pages are async server components that call `requireRole` and read through RLS. Mutations are server actions ending in `revalidatePath`. There is no client data library.
- **Dependencies.** `app/` → `src/features/` → `src/data/`, `src/domain/`, `src/ui/`. `src/domain/` imports nothing, so its rules are unit tested without a database.

Errors. Database functions raise `TLnnn:REASON:detail`. `callRpc` maps the reason to fixed copy with `parseDbError` (`src/lib/errors.ts`) and logs blocked attempts with `log_denied`, so the UI shows one consistent sentence and never a raw database message.

Logging. `src/lib/logger.ts` writes one JSON line per event to Vercel logs and redacts fields whose names match `password|token|secret|key|authorization|cookie|text|email`.

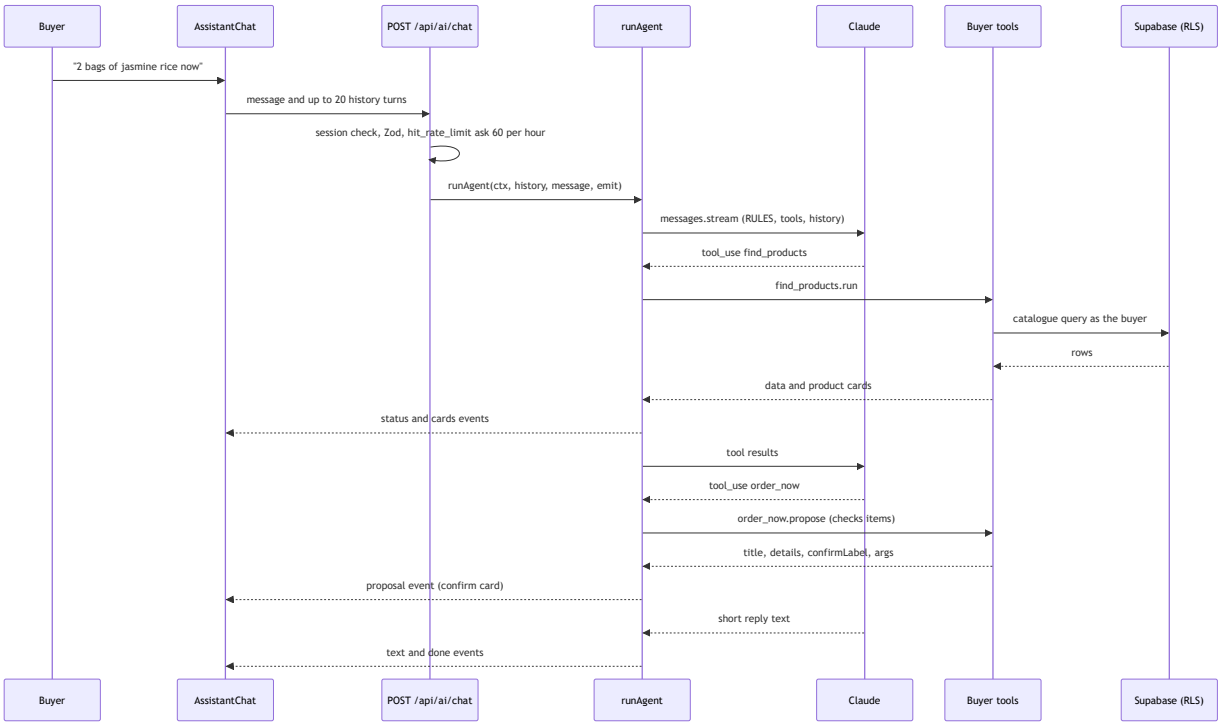
5. Key flows

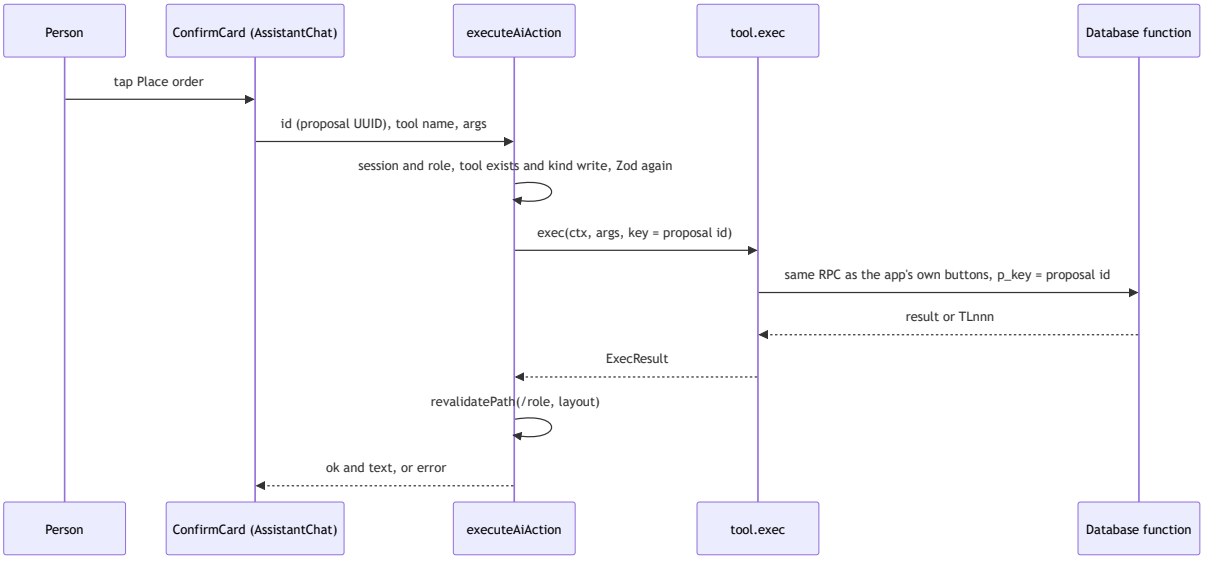
5.1 Sign-in and roles



A user belongs to one organisation, and the organisation's `kind` is the role; the same fact drives RLS. The proxy only redirects as a convenience; the database enforces every permission. Self sign-up runs the trigger `app.handle_new_user`, which creates an active organisation and connects it to every active counterpart.

5.2 Tradelink AI with confirm cards



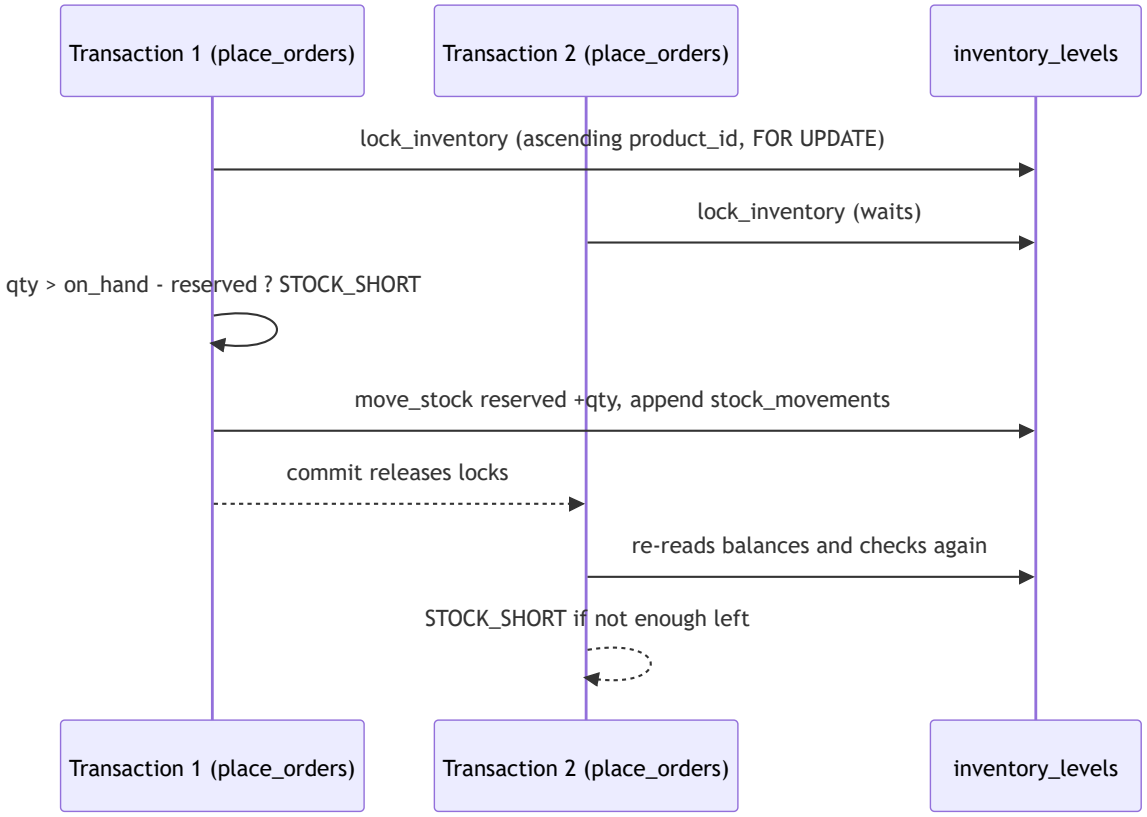


- **Why this design.** The model only reads and proposes. A change runs only after a person taps, is validated again on the server, and goes through the same database functions as the app's own buttons, so RLS and role checks apply unchanged. The proposal id doubles as the idempotency key, so a double tap is harmless.
- **Limits.** Message 1 to 2,000 characters; 20 history turns; 60 messages per hour per user; at most 6 tool rounds and 900 output tokens per round; effort `low`. Prompt caching covers the rules, the tool list and the conversation.
- **Tools.** Buyers: 11 read, 17 confirm, 2 card-only. Suppliers: 7 read, 17 confirm, 1 card-only (`buyer-tools.ts`, `supplier-tools.ts`).

- **Prompt injection.** Tool results are treated as data, tools run as the user under RLS, IDs are resolved on the server, and writes need a tap (ADR 0010).
- **Failure.** No key gives HTTP 503 "Tradelink AI is not set up yet."; an exception gives "Tradelink AI is not available right now."; there is no rules fallback on this path.

5.3 Placing orders and reserving stock

`place_orders` merges duplicate lines, checks visibility, locks inventory, checks availability, creates one order per supplier, reserves stock, and writes order events, the audit log and a "New order" notification, all in one transaction. A null draft version means a direct order ("Order now" or "Place tomorrow's order"), which leaves the cart alone.

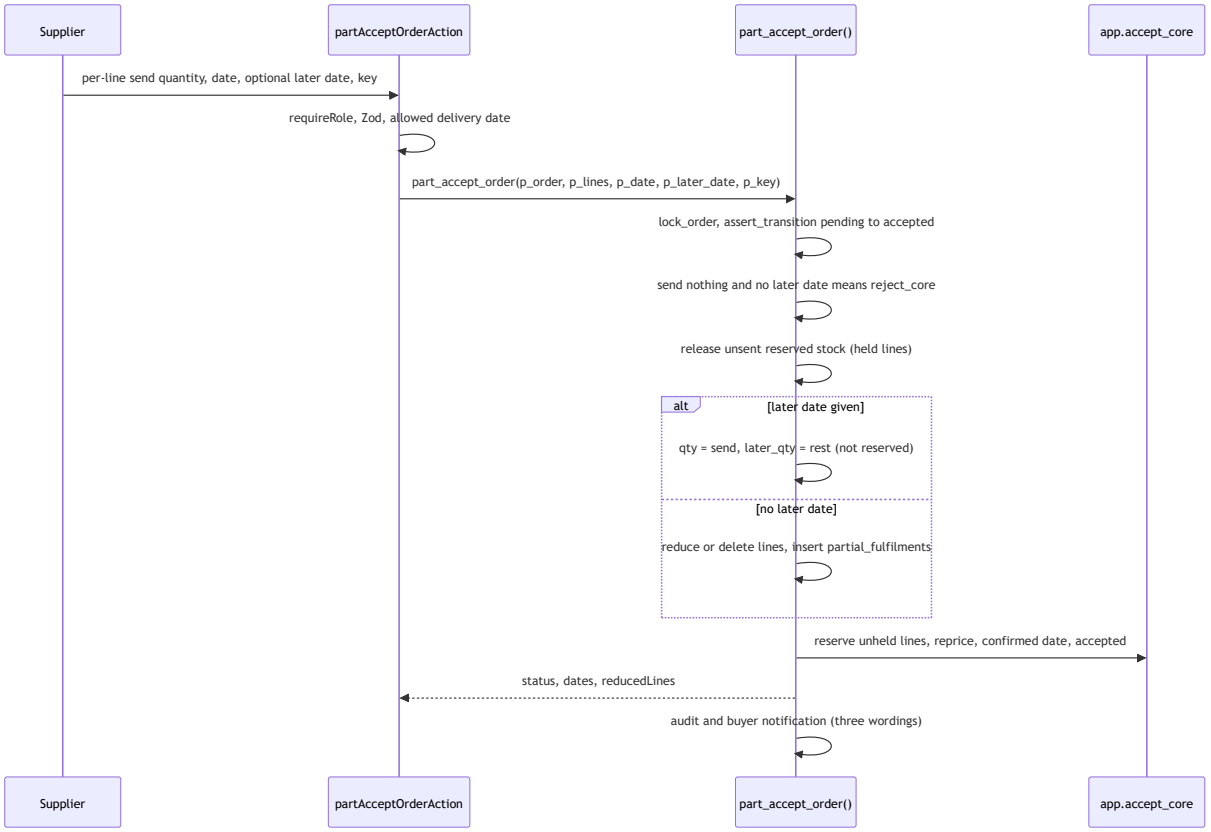


`supabase/migrations/20261103085139_app_functions.sql`

```
create function app.lock_inventory(p_products uuid[]) returns void
language plpgsql set search_path = '' as $$
begin
  perform 1 from public.inventory_levels i
  where i.product_id = any (p_products)
  order by i.product_id
  for update;
end $$;
```

Why. Checking and reserving in one transaction under row locks is the simplest way to guarantee no overselling (ADR 0006). Locking in a fixed order prevents deadlocks between stock functions, and check constraints (`reserved <= on_hand`, both non-negative) are the backstop. Every change appends a `stock_movements` row, so balances are explainable.

5.4 Part acceptance and delivery



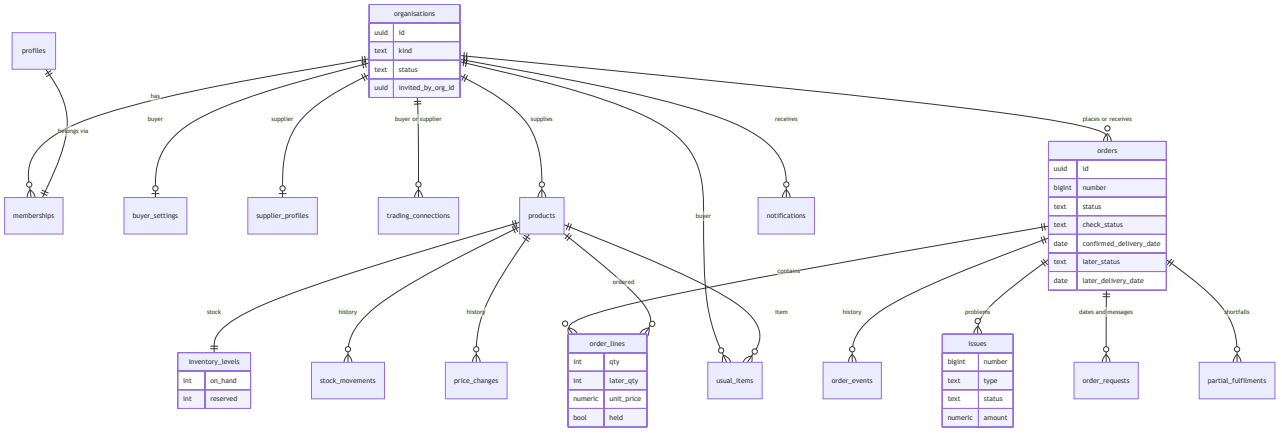
The rest of a part-accepted order is not reserved (Inferred rationale: the stock is not there yet, and reserving it would break `reserved <= on_hand`). `deliver_rest` checks free stock when the rest goes and fails with `REST_NOT_IN_STOCK` if it is not there.

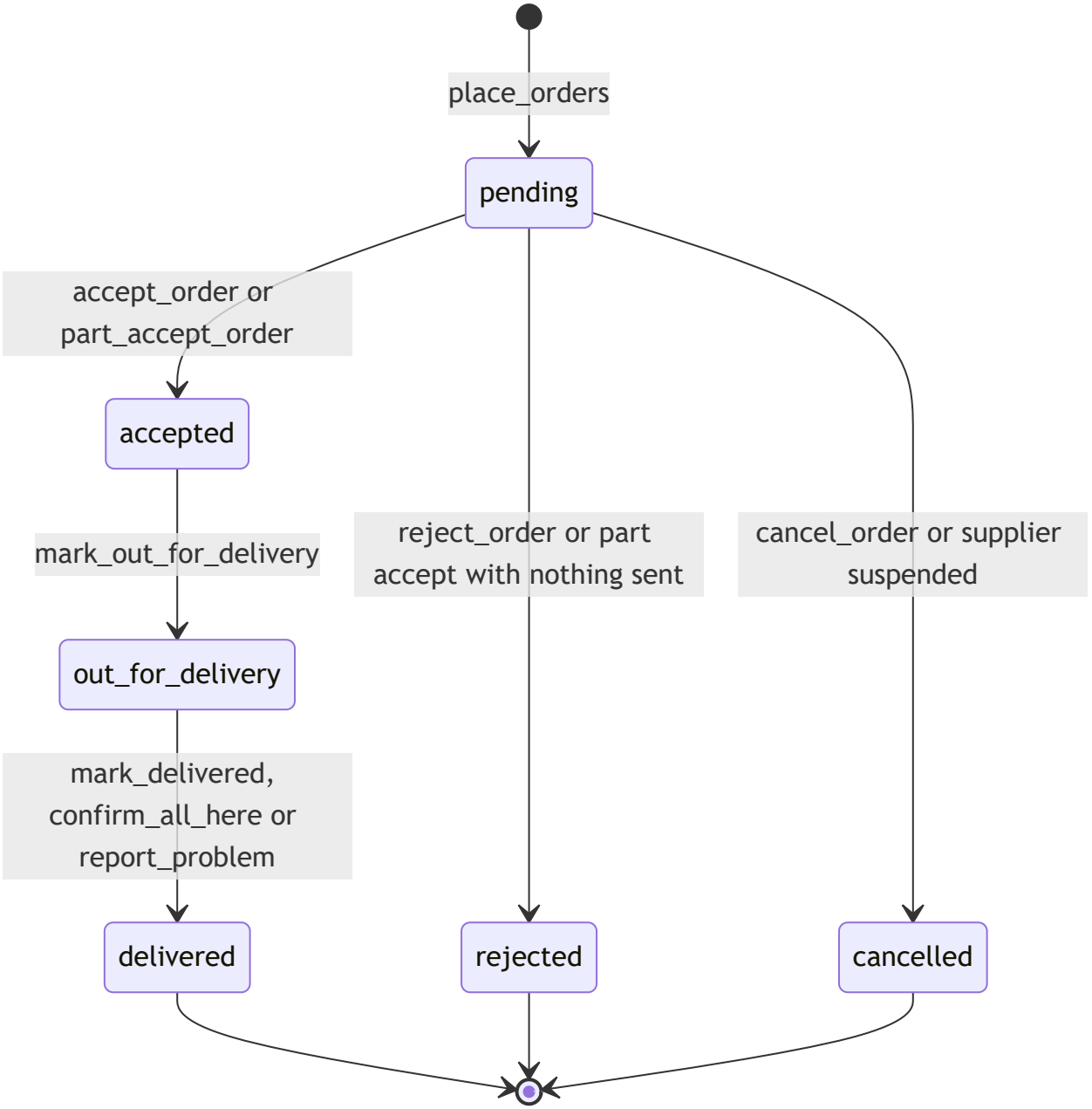
5.5 Delivery, problems, credits and disputes

- `mark_out_for_delivery`, then `mark_delivered` (`deliver_order` removes the quantity from both `on_hand` and `reserved`).
- The buyer confirms with `confirm_all_here`, or calls `report_problem`, which delivers first if the order is still out and records the amount as quantity × unit price.
- The supplier answers with `answer_issue`: credit, replace, on the way (late only) or dispute. A dispute notifies admins and becomes a high-priority attention item, settled with `resolve_dispute`.
- A promised credit stays open until the buyer calls `mark_credit_received`, which tells the supplier.

Notifications are in-app rows written by `app.notify` inside the same transaction as the change, so a notification never describes a change that rolled back. The bell panel reads them through RLS and picks a button (Reply, Answer, Confirm credit) per kind.

6. Data model


profiles
has
belongs via
organisations
uuid id
text kind
text status
uuid invited_by_org_id
buyer
supplier
buyer or supplier
supplies
receives
places or receives
memberships
buyer_settings
supplier_profiles
trading_connections
products
buyer
notifications
orders
uuid id
bigint number
text status
text check_status
date confirmed_delivery_date
text later_status
date later_delivery_date
contains
stock
history
history
ordered
item
history
problems
dates and messages
shortfalls
inventory_levels
int on_hand
int reserved
stock_movements
price_changes
order_lines
int qty
int later_qty
numeric unit_price
bool held
usual_items
order_events
issues
bigint number
text type
text status
numeric amount
order_requests
partial_fulfilments


place_orders
pending
accept_order or
part_accept_order
accepted
mark_out_for_delivery
out_for_delivery
mark_delivered,
confirm_all_here or
report_problem
delivered
reject_order or part
accept with nothing sent
rejected
cancel_order or supplier
suspended
cancelled

Rule	Enforced by
Status moves only along allowed edges (orders, issues, organisations)	`app.state_transitions` table plus the `app.guard_status` trigger; functions call `app.assert_transition` first for a friendly message (ADR 0007)
Reserved stock never exceeds stock on hand	Check constraints, plus checks under lock in `place_orders` and `accept_core`
History cannot be edited	`app.forbid_change` triggers on `audit_log`, `stock_movements`, `price_changes`, `order_events`
One open "late" problem per order	Unique index `issues_one_open_late`
Schema changes	A new file in `supabase/migrations/`, named with the applied version; applied migrations are never edited

The 17 migrations also create about 40 `rs_*` tables for a "Tradelink AI shop" foundation (cards, jobs, push). Only the `/r` page uses them (section 12).

7. Security

Area	How it is handled
Authentication	Supabase Auth with cookie sessions (`@supabase/ssr`); no two-factor sign-in (ADR 0024)
Authorisation	`requireRole` on pages and actions; RLS on every table; every write function calls `app.require_kind` or `app.require_member`; the operations lead is `admin_profiles.is_lead`
Data isolation	Tenancy by organisation; another business's order raises the same `NOT_FOUND` as a missing one
Secrets	`ANTHROPIC_API_KEY`, `SUPABASE_SECRET_KEY` and `VAPID_PRIVATE_KEY` are server only, in `.env.local` (git-ignored) and Vercel; the demo password is not in the repository
Input validation	Zod in every action and route, argument checks in database functions, table constraints
Rate limits	Postgres fixed window per user: AI chat 60 per hour, order messages 30 per hour
Audit	Every write calls `app.audit`; blocked attempts are logged; the log is append-only
Headers	`X-Frame-Options: DENY`, `nosniff`, HSTS, Referrer-Policy, Permissions-Policy; no CSP
File uploads	None

Accepted risks: no two-factor sign-in; self sign-up connects a new business to every counterpart; `/api/jobs/run` is public (runs only due jobs, returns counts); a demo password remains in old commits (owner decision, 4 November 2026).

8. Testing

Suite	Command	Result on 4 November 2026
Unit (Vitest, 72 files)	`npm test`	**838 passed, 1 skipped** (71 files passed, 1 skipped; 33 s)

Suite	Command	Result on 4 November 2026
End to end, read only	`npx playwright test e2e/smoke.spec.ts e2e/pages.spec.ts`	**16 passed** (8 tests on desktop Chrome and on a Pixel 7 profile)
End to end, order flow	`npx playwright test e2e/order-flow.spec.ts`	Not run: writes real orders to the live database
Stock concurrency (24 parallel orders)	`RUN_DB_TESTS=1 NODE_OPTIONS=--experimental-websocket npm test -- tests/concurrency`	Not run: writes to the live database
Coverage	—	Not measured: no coverage tool
Type-check and lint	`npm run typecheck`, `npm run lint`	Clean

End-to-end tests read the demo password from `E2E_PASSWORD` in `.env.local`. Tests that write should run against a separate test project, which needs Supabase Pro for branching (section 3).

9. Delivery and operations

- **Git and releases.** Work happens on `develop`; release by fast-forwarding `main` (`git push origin develop:main`). There is no CI; run `npm run check` before pushing.
- **Vercel.** The GitHub repository is connected: `develop` makes previews, `main` deploys production. Environment variables live in the Vercel project, server-only values marked sensitive.
- **Database.** Test a migration in a rolled-back transaction, apply it to Supabase, then save it as `supabase/migrations/<version>_<name>.sql`. Database changes go live before the app deploys, so keep them backward compatible.
- **Rollback.** App: promote the previous production deployment in Vercel, or revert on `main`. Database: a new migration that restores the earlier definition.
- **Monitoring.** `GET /api/health` is liveness only. Structured logs go to Vercel; `ai_usage` records AI tokens and latency. No alerting and no scheduled jobs.
- **Documentation tooling.** `npm run docs:screenshots` recaptures the guide's screenshots; `npm run docs:pdf` rebuilds the PDFs and diagrams.

10. Running cost

Measured AI usage (from `public.ai_usage`, 3 to 4 November 2026, model `claude-sonnet-5-5`):

Feature	Successful calls	Average input tokens	Average output tokens	Total tokens (in / out)
Tradelink AI chat (`ask`)	135	6,551 (includes cached tokens)	223	884,350 / 30,104
Order reader (unused path)	5	2,889	168	14,447 / 840

Price. Claude Sonnet 5.5 list price: US$2 per million input tokens, US$10 per million output tokens, US$0.20 per million cache-read tokens (Anthropic model documentation).

- **Cost per chat reply (upper bound):** 6,551 × US$2 / 1,000,000 + 223 × US$10 / 1,000,000 ≈ US$0.013 + US$0.002 ≈ **US$0.015**. This charges every input token at the full rate; cache reads cost a tenth, so the real figure is lower.
- **The 135 measured replies** cost at most 884,350 × US$2 / 1,000,000 + 30,104 × US$10 / 1,000,000 ≈ US$1.77 + US$0.30 ≈ **US$2.07**.

Hosting today. Vercel Hobby (free, for personal and non-commercial use; the Vercel API reports "hobby plan") and Supabase Free (branching failed because it needs Pro). Supabase Free pauses a project after 7 idle days and has a 500 MB database (third-party pricing summary), which is a reason to move to Pro before real customers.

Hosting for commercial use. Vercel Pro US$20 per developer seat per month; Supabase Pro US$25 per month per organisation. These list prices come from third-party pricing summaries (September to November 2026); check vercel.com/pricing and supabase.com/pricing before buying.

Illustrative monthly budget (estimate, not measured):

Scenario	Hosting	AI (replies × US$0.015)	Total per month
Pilot	Free plans: US$0	1,000 × US$0.015 ≈ US$15	**≈ US$15**
Small launch	Vercel Pro 1 seat US$20 + Supabase Pro US$25 = US$45	6,000 × US$0.015 ≈ US$92	**≈ US$137**

Usage-based extras (Vercel and Supabase overages, extra seats) are not included.

11. Decision log

Product decisions are in the Functional Specification as DEC-xx. Same decision in both documents: TD-01 = DEC-01, TD-02 = DEC-02, TD-03 = DEC-02, TD-05 = DEC-03, TD-06 = DEC-05, TD-08 = DEC-04, TD-10 = DEC-06, TD-12 = DEC-08, TD-18 = DEC-09, TD-19 = DEC-11.

ID	Decision	Why	Trade-off	Record
TD-01	Responsive PWA instead of native apps	One codebase for phone and laptop	No app store presence	ADR 0016
TD-02	Next.js App Router on Vercel	Keys and AI stay on the server; branch previews	Tied to Vercel runtime limits (60 s route)	ADR 0002
TD-03	Supabase for Postgres, Auth and RLS	Transactions and RLS fit stock and tenancy	Logic in SQL is harder to unit test	ADR 0003
TD-04	Writes only through definer functions	Role, state, stock, audit and notification in one transaction	More SQL to maintain	ADR 0004

ID	Decision	Why	Trade-off	Record
TD-05	Server checks plus RLS	Clear redirects; the database enforces for every caller, including the AI	Two places to keep in sync	ADR 0005
TD-06	Stock reserved in the order transaction with ordered locks	No overselling, no deadlocks between stock functions	Lock waits under contention	ADR 0006
TD-08	Every AI change needs a person's tap	People stay in control; same database functions apply	One extra tap	ADR 0008
TD-09	Tool-using agent instead of grounded-facts Ask	**Inferred rationale:** the AI must be able to do what users can do	ADR 0023 is out of date	`src/features/ai/agent.ts`
TD-10	Self sign-up, no 2FA, instead of token invitations	Owner decision for a working demo	Invited suppliers have no join path	ADR 0024
TD-12	Light and dark mode from design tokens	One token file for both themes	Token changes need `npm run tokens`	ADR 0015
TD-13	Theme in a per-browser cookie	**Inferred rationale:** the server renders the right theme before paint	Does not follow the person across devices	`src/ui/ThemeToggle.tsx`
TD-18	One order per supplier from one cart	Each supplier handles its own order	Several order numbers per checkout	`place_orders`
TD-19	Operations lead sees and assigns all attention items	Owner request, 4 November 2026	One lead flag, no team management UI	`20261104040939_attention_lead_sees_all.sql`
TD-20	The second delivery is not reserved	**Inferred rationale:** the stock is not there yet	Can fail later with `REST_NOT_IN_STOCK`	`deliver_rest`

ID	Decision	Why	Trade-off	Record
TD-21	Quick orders use one confirm tap, not the cart	**Inferred rationale:** a known short order should take one tap	Less review than the cart	`order_now`

For owner review: the inferred rationales above (TD-09, TD-13, TD-20, TD-21), a new ADR for the tool-using agent, and whether the 21 ADRs still marked "Proposed" should become "Accepted".

12. Known technical debt and possible defects

Technical debt

- Unused code: the Claude order reader and grounded Ask (`src/integrations/claude/read-order.ts`, `src/domain/reader/`, `src/features/ask/`), the unwired `render_card` tool, and the `rs_*` Tradelink AI foundation used only by `/r`.
- Built but not live: push notifications, the job runner (no cron).
- i18n dictionaries are used only in PWA messages; the theme is per browser (`profiles.theme` unused).
- No CI, coverage, accessibility tests, pgTAP or AI evaluations.
- Log redaction is by field name only; request and user IDs are not added automatically.

Possible defects (recorded, not fixed)

ID	Finding	Suggested next step
BUG-01	Delivering "the rest" before the first delivery counts stock twice (`deliver_rest` allowed while accepted)	Allow it only after the first delivery
BUG-02	An invited supplier signing up under the same business name is blocked as a duplicate	Decide on a join flow
BUG-03	Supplier answers for "invoice" problems exist, but nothing creates one	Remove the remnants or build invoice queries
SEC-01	Later `app.rs_*` helpers keep the default PUBLIC execute grant; later tables lack explicit revokes (RLS still blocks writes)	Add revokes in a new migration
SEC-02	Any user can call `hit_rate_limit` with any action name; rows are never cleaned up	Restrict names; add cleanup
SEC-03	No Content Security Policy header	Add a CSP
OPS-01	`/api/jobs/run` is public and nothing calls it	Schedule it or remove it
OPS-02	`set_supplier_status` and `cancel_order` lock rows in opposite orders; a rare deadlock is possible	Lock in one order

Appendix A. Differences from older documents

The code is authoritative; older documents were not changed.

Document	Says	As built
`docs/DATA_MODEL.md`	Write functions in an `api` schema	In `public`, helpers in `app`
`docs/DATA_MODEL.md`	Invoice, invitation, negotiation and email tables	Not created
`docs/DATA_MODEL.md`, `docs/SECURITY.md`	Admins need two-factor (aal2)	Removed in `20261103104222_self_sign_up.sql`
`docs/SECURITY.md`	Open sign-up is off	Self sign-up exists
`docs/RPC_CONTRACT.md`	`v_attention` has ten rules	8 causes
`docs/API.md`	Invoice and invitation actions; `/api/health/ready`	Not built
`docs/ARCHITECTURE.md`	Email outbox with Resend; `/api/ask` with server-sent events	Not built; the assistant is `/api/ai/chat` streaming NDJSON
`docs/TEST_PLAN.md`	pgTAP, axe, AI evals, Lighthouse CI, 90% coverage gate	None exist
ADR 0010	The model has no write tools; strict CSP	Write tools gated by confirm cards; no CSP
ADR 0023	Ask is not a tool runner	The live assistant is a tool runner

Document history

Version	Date	Author	Change
1.0	4 November 2026	Prepared for Janya Mali	First as-built technical specification, from the code at `9250e98` and 17 applied migrations
1.1	4 November 2026	Prepared for Janya Mali	Condensed to the essentials; added technology choices, options not used, and running cost